**Отчет по изменению**
**Индивидуального Домашнего Задания №2**
**Вариант 35**

Сделано
Жарковой Татьяной Владимировной
студенткой 2 курса Программной инженерии
группы БПИ-253

Москва 2026
Высшая школа экономики

# Оглавление


**Аннотация** **4**

**Изменения в изначальной версии** **5**

Изменение `model.h` 5

Потоки сущностей 5

Многопоточность `Simulation` 6

Изменения `dispatch.h` 7

Добавление структуры `Assignment` 7

Добавление функции `plan_dispatch()` 7

Удаление файла `event_handlers.h` 8

Изменение в `simulation.h` 8

Удалена функция `cancel_or_unserve_expired()` 8

Удалена функция `install_signal()` 8

Удалена функция `stop_is_requested()` 8

Механизм сообщения потоков `sync.h` 8

Типы сообщений 8

Буфер сообщений `Mailbox` 10

Счетчик `Quiet` 10

Управление потоками `threads.h` 12

Создание файла `threads_internal` 13

Изменение `utils_extra.h` 13

Добавлена функция `set_thread_tag()` 13

Добавлена функция `output_begin()` 14

Добавлена функция `output_end()` 14

Изменение `dispatch.c` 14

Функция `plan_dispatch()` 14

Изменение `generator.c` 15

Изменение `find_free_call_slot()` 15

Изменение `utils.c` 15

Добавление собственной копии переменной потока 15

Добавление функции `set_thread_tag()` 16

Добавление функции `output_begin()` 16

Добавление функции `output_end()` 16

Изменение `log_event()` 17

Изменение `simulation.c` 17

Добавление `read_control()` 17

Добавление `snapshot_locked` 18

Добавление `spawn_random_call()` 18

Добавление `deliver()` 19

Изменение `run_smulation()` 19

Новый файл `threads.c` 20

Функция `send_message()` 20

Функция `send_shutdown()` 20

Функция `create_thread()` 21
Функция `init_thread_sync()` 21
Функция `destroy_thread_sync()` 22
Функция `start_service_threads()` 22
Функция `start_call_thread()` 23
Функция `join_call_thread()` 24
Функция `stop_all_threads()` 24
Файл `call_threads.c` 24
Функция `call_on_event()` 24
Функция `call_thread()` 25
Файл `dispatcher_thread.c` 25
Функция `dispatcher_thread()` 25
Файл `signal_threads.c` 26
Функция `fill_signall_set()` 26
Функция `block_signals` 27
Функция `signal_thread()` 27
Файл `team_thread.c` 27
Функция `team_thread()` 27
Файл `sync_sem.c` 28
Функция `sem_wait_retry()` 28
Функция `mailbox_init()` 29
Функция `mailbox_put()` 30
Функция `mailbox_get()` 30
Функция `quiet_init()` 31
Функция `quiet_destroy()` 31
Функция `quiet_add()` 32
Функция `quiet_done()` 32
Функция `quiet_wait()` 32
Файл `sem_cv.c` 33
Функция `mailbox_init()` 33
Функция `mailbox_destroy()` 33
Изменение `main.c` 38

## Аннотация

В данном отчете представлена работа над ошибками индивидуального домашнего задания, вариант 35. В силу того, что было изменено 1 задание. Ниже будут описаны важные изменения оригинальной программы (отчитываясь с 1ого задания).

## Изменения в изначальной версии

### Изменение `model.h`

**Потоки сущностей**

```c
/**
 * @struct Team
 * @brief Team in the simulation
 */
typedef struct
{
    int id;
    Specialization spec;
    int speed;
    int district;
    int base;
    TeamState state;
    int call_id;

    //thread of the team
    pthread_t thread;
    Mailbox mailbox; // commands from the dispatcher and wake-ups from the clock
    int started;     // 1 if the thread was created

} Team;
```

- `pthread_t thread` - идентификатор потока бригады;
- `Mailbox mailbox`- почта бригады, то есть это механизм передачи сообщений между потоками;
- `int started` - флаг был ли поток создан и не присоединен ли он уже к основному потоку через `pthread_join()`.

Аналогично с потоками вызова:

```c
/**
 * @struct Call
 * @brief Call in the simulation
 */
typedef struct
{
    int id;
    int district;
    int x, y; // coordinates of the call location
    int spec;
    int priority;
    int arrival;
    int service_duration;
    int max_wait;
    int escalation_after;
    int cancel_after;
    CallState state;
    int assigned_team;
    int dispatch_time;
    int service_start;
    int completion_time;

    // thread of the call
    pthread_t thread;
    Mailbox mailbox; // wake-ups from the clock and the message of the team
    int started;     // 1 if the thread was created

} Call;
```

**Многопоточность `Simulation`**

```c
    pthread_mutex_t world;       // protects the whole model: teams, calls, event queue, statistics
    pthread_mutex_t output_lock; // one output line (or box) is printed by one thread at a time
    pthread_mutex_t control;     // protects the flags below, they are set by the signal thread
    int stop_requested;          // SIGINT / SIGTERM
    int close_requested;         // SIGQUIT
    int snapshot_requested;      // SIGUSR1
    int shutting_down;           // the signal thread must finish

    Quiet quiet;                 // counter of not yet handled messages
    Mailbox dispatcher_box;      // mailbox of the dispatcher thread
    pthread_t dispatcher;
    pthread_t signal_thread;
    int threads_started;         // number of created threads (for the summary)

} Simulation;
```

- `world` - защищает общее состояние модели;
- `output_lock` - не позволяет потокам перемешивать вывод;
- `control` - защищает флаги управления сигналами;

- `stop_requested` - запрос немедленной остановки;
- `close_requested` - запрос закрытия приёма новых вызовов;
- `snapshot_requested` - запрос снимка состояния;
- `shutting_down` - сообщает потоку сигналов о завершении;
- `quiet` - отслеживает количество необработанных сообщений;
- `dispatcher_box` - почтовый ящик диспетчера;
- `dispatcher` - поток диспетчера;
- `signal_thread` - поток обработки сигналов.

## Изменения `dispatch.h`

**Добавление структуры `Assignment`**

Для удобства работы

```
/**
 * @struct Assignment
 * @brief Decision of the dispatcher: team index -> call index
 */
typedef struct
{
    int team;
    int call;
} Assignment;
```

**Добавление функции `plan_dispatch()`**

Удалена функция `try_dispatch()` и заменена на `int plan_dispatch(Simulation *sim, Assignment *out, int max)`.

Теперь появляются 2 обязанности:

1. Диспетчер принимает решение о назначении;
2. Поток бригады выполняет назначение.

Эта функция не выполняет поездку за бригаду. Она только формирует решения.

## Удаление файла `event_handlers.h`

Ранее этот файл занимался обработкой событий. Однако теперь этой задача делегирована потокам.

- события вызовов обрабатываются через `call_on_event()`;
- события бригад — через `team_on_event()`;
- распределение — через `dispatcher_thread()`.

## Изменение в `simulation.h`

**Удалена функция `cancel_or_unserve_expired()`**

Задача передана конкретному потоку вызова.

**Удалена функция `install_signal()`**

Используется другая функция `block_signals()`.

**Удалена функция `stop_is_requested()`**

Теперь состояние остановки хранится в `sim->stop_requested` и защищается мьютексом `control`.

## Механизм сообщения потоков `sync.h`

**Типы сообщений**

```c
/**
 * @enum MsgType
 * @brief Kinds of messages that threads send to each other
 */
typedef enum
{
    MSG_WAKE = 0,      // clock -> call/team: an event of this entity has come (Message.event)
    MSG_NEW_CALL,      // call -> dispatcher: the call has arrived and waits (arg = call index)
    MSG_CALL_CHANGED,  // call -> dispatcher: the priority of a waiting call has changed
    MSG_TEAM_FREE,     // team -> dispatcher: the team is available again (arg = team index)
    MSG_ROUND,         // clock -> dispatcher: all reactions of this minute are done, assign teams
    MSG_ASSIGN,        // dispatcher -> team: serve the call (arg = call index)
    MSG_CALL_FINISHED, // team -> call: the help is finished, the call thread can end
    MSG_SHUTDOWN       // clock -> everybody: the simulation is over
} MsgType;
```

- `MSG_WAKE` - наступило событие;
- `MSG_NEW_CALL` - поступил новый вызов;
- `MSG_CALL_CHANGED` - изменился приоритет;
- `MSG_TEAM_FREE` - бригада освободилась;
- `MSG_ROUND` - начинается распределение;
- `MSG_ASSIGN` - назначен вызов;
- `MSG_CALL_FINISHED` - помощь завершена;
- `MSG_SHUTDOWN` - завершение работы;

Каждое сообщение имеет структуру:

```c
/**
 * @struct Message
 * @brief One message in a mailbox
 */
typedef struct
{
    MsgType type;
    int event; // EventType for MSG_WAKE
    int arg;   // index of a call or a team
} Message;
```

- `type` - тип сообщения;
- `event` - тип события, если это пробуждение;
- `arg` - дополнительный индекс вызова или бригады.

**Буфер сообщений `Mailbox`**

```c
/**
 * @struct Mailbox
 * @brief Bounded FIFO queue of messages of one thread
 */
typedef struct
{
    Message *items;
    int capacity;
    int head;  // index of the oldest message
    int count; // number of messages in the box
    pthread_mutex_t lock;

#ifdef USE_CONDVAR
    pthread_cond_t not_empty;
    pthread_cond_t not_full;
#else
    sem_t filled; // number of messages that can be received
    sem_t free;   // number of free places
#endif

} Mailbox;
```

- `items` - массив сообщений;
- `capacity` - вместимость;
- `head` - индекс первого сообщения;
- `count` - количество сообщений;
- `lock` - защищает структуру буфера.

Если определен макрос `USE_CONDVAR`, используются условные переменные, иначе используются семафоры.

**Счетчик `Quiet`**

```c
/**
 * @struct Quiet
 * @brief Counter of messages that were sent but are not fully handled yet
 */
typedef struct
{
    pthread_mutex_t lock;
    int pending;
#ifdef USE_CONDVAR
    pthread_cond_t zero;
#else
    sem_t idle; // posted exactly once when pending drops to 0
#endif
} Quiet;
```

Это счетчик незавершенных сообщений. Он нужен часам, чтобы определить, когда текущая фаза симуляции закончена.

```c
// Initializes a mailbox with the specified capacity
int mailbox_init(Mailbox *box, int capacity);

// Releases all resources allocated for the mailbox
void mailbox_destroy(Mailbox *box);

// Adds a message to the mailbox, blocking if it is full
int mailbox_put(Mailbox *box, Message message);

// Retrieves a message from the mailbox, blocking if it is empty
Message mailbox_get(Mailbox *box);

// Initializes the synchronization mechanism for tracking pending operations
int quiet_init(Quiet *quiet);

// Releases all resources allocated for the Quiet synchronization object
void quiet_destroy(Quiet *quiet);

// Increments the counter of pending operations
void quiet_add(Quiet *quiet);

// Decrements the counter of pending operations and signals when it reaches zero
void quiet_done(Quiet *quiet);

// Blocks until all pending operations have been completed
void quiet_wait(Quiet *quiet);

// Returns the name of the synchronization mechanism used by the program
const char *sync_flavour(void);
```

Управление потоками `threads.h`

```c
#ifndef THREADS_H
#define THREADS_H

#include "model.h"

#define ENTITY_BOX_SIZE 16 // mailbox of a call or a team
#define DISPATCHER_BOX_SIZE (2 * MAX_CALLS + MAX_TEAMS + 16) // mailbox of the dispatcher

// Initializes all synchronization objects required by the simulation threads
int init_thread_sync(Simulation *sim);

// Destroys all synchronization objects and releases associated resources
void destroy_thread_sync(Simulation *sim);

// Blocks signals that will be handled by the dedicated signal thread
int block_signals(void);

// Creates and starts the dispatcher, signal, and team threads
int start_service_threads(Simulation *sim);

// Creates and starts a separate thread for the specified call
int start_call_thread(Simulation *sim, int call_idx);

// Waits for the specified call thread to finish
int join_call_thread(Simulation *sim, int call_idx);

// Requests termination of all threads and waits for their completion
void stop_all_threads(Simulation *sim);

// Sends a message to a mailbox and updates the pending operation counter
void send_message(Simulation *sim, Mailbox *box, MsgType type, int event, int arg);

#endif
```

Основная задача этого файла - управление жизненным циклом многопоточной системы.

- `init_thread_sync()` - инициализация мьютексов, почтовых ящиков и других объектов синхронизации;
- `destroy_thread_sync()` - освобождение ресурсов синхронизации;
- `block_signals()` - блокировка сигналов для последующей обработки отдельным потоком;
- `start_service_threads()` - создание и запуск служебных потоков;
- `start_call_thread()` - создание потока для конкретного вызова;
- `join_call_thread()` - ожидание завершения потока вызова;
- `stop_all_threads()` - организация завершения работы потоков;
- `send_message()` - отправка сообщения в почтовый ящик.

Создание файла `threads_internal`

```c
#ifndef THREAD_INTERNAL_H
#define THREAD_INTERNAL_H

#include "threads.h"

/**
 * @struct ThreadArgs
 * @brief Arguments passed to a call or team thread.
 */
typedef struct
{
    Simulation *sim;
    int index;
} ThreadArgs;

// Thread functions for call
void *call_thread(void *raw);

//Thread function for a team thread
void *team_thread(void *raw);

// Thread functions for dispatcher
void *dispatcher_thread(void *raw);

// Thread function for signal handling
void *signal_thread(void *raw);
```

Этот файл содержит объявления, необходимые нескольким исходным файлам многопоточной реализации:

- `ThreadArgs` - структура аргументов, передаваемых потокам;
- `call_thread()` - функция потока вызова;
- `team_thread()` - функция потока бригады;
- `dispatcher_thread()` - функция потока диспетчера;
- `signal_thread()` - функция потока обработки сигналов.

Изменение `utils_extra.h`

**Добавлена функция `set_thread_tag()`**

`void set_thread_tag(const char *tag);`

Позволяет обозначить поток в журнале.

**Добавлена функция `output_begin()`**
Блокирует вывод.

**Добавлена функция `output_end()`**
Освобождает вывод.

## Изменение `dispatch.c`

**Функция `plan_dispatch()`**

Теперь функция должна вернуть список назначений, а не самостоятельно выполнять их.

Теперь бригада самостоятельно:

- получает назначение;
- рассчитывает время поездки;
- планирует событие прибытия.

```
    Call *call = &sim->calls[selected];

    int chosen_team = choose_team(sim, call);
    SOFT_ASSERT(chosen_team >= 0 && chosen_team < sim->team_count, "selected team index is invalid", made);

    Team *team = &sim->teams[chosen_team];

    // reserve the team and the call
    call->assigned_team = team->id;
    call->dispatch_time = sim->now;
    call->state = CALL_EN_ROUTE;
    team->state = TEAM_EN_ROUTE;
    team->call_id = call->id;

    remove_call_deadlines(sim, call->id); // the call does not wait any more

    sim->dispatches++;

    log_event(sim, "DISPATCH: CALL #%03d priority=%d spec=%s waited %d min -> B%d (%s strategy)", call->id,
              call->priority, spec_name(call->spec), sim->now - call->arrival, team->id, strategy_name(sim->strategy));

    out[made].team = chosen_team;
    out[made].call = selected;
    made++;
}

return made;
```

## Изменение `generator.c`

### Изменение `find_free_call_slot()`

```
    for (int idx = 0; idx < sim->call_count; idx++)
    {
        CallState state = sim->calls[idx].state;
        int closed = state == CALL_COMPLETED || state == CALL_CANCELLED || state == CALL_UNSERVED;

        // the thread of a closed call is idle, it is stopped and joined
        if (closed)
        {
            join_call_thread(sim, idx);
            return idx;
        }
    }
```

Так как у каждого вызова есть свой поток, то мы не можем просто переиспользовать структуру. Теперь даже если вызов уже завершен с точки зрения модели, его поток еще может существовать. Поэтому сначала мы делаем `join` и после этого перезаписываем структуру.

## Изменение `utils.c`

### Добавление собственной копии переменной потока

`static _Thread_local char thread_tag[16] = "main";`

Если бы мы использовали обычную глобальную переменную, потоки перезаписывали бы её друг другу. Поэтому тут мы используем `_Thread_local`.

**Добавление функции `set_thread_tag()`**

```c
/**
 * @brief Sets the name of the current thread for the log
 * @param tag Short name
 */
void set_thread_tag(const char *tag)
{
    if (tag == NULL)
        return;

    snprintf(thread_tag, sizeof(thread_tag), "%s", tag);
}
```

Функция записывает имя текущего потока.

**Добавление функции `output_begin()`**

```c
/**
 * @brief Locks the console/log output
 */
void output_begin(Simulation *sim)
{
    if (sim)
        pthread_mutex_lock(&sim->output_lock);
}
```

Эта функция блокирует вывод.

**Добавление функции `output_end()`**

```c
/**
 * @brief Unlocks the console/log output
 */
void output_end(Simulation *sim)
{
    if (sim)
        pthread_mutex_unlock(&sim->output_lock);
}
```

Эта функция разблокировывает вывод.

**Изменение `log_event()`**

```c
// the same order of lines in the console and in the log file
pthread_mutex_lock(&sim->output_lock);

write_all(STDOUT_FILENO, line, (size_t)len);

if (sim->log_fd >= 0)
    write_all(sim->log_fd, line, (size_t)len);

pthread_mutex_unlock(&sim->output_lock);

return 1;
```

- Теперь у каждого сообщения появляется обозначение потока;
- Вывод защищается `mutex`.

## Изменение `simulation.c`

`simulation.c` определяет модельное время и организует работу остальных потоков.

**Добавление `read_control()`**

```c
/**
 * @brief Reads (and resets) the requests of the signal thread
 */
static void read_control(Simulation *sim, int *stop, int *close, int *snapshot)
{
    pthread_mutex_lock(&sim->control);
    *stop = sim->stop_requested;
    *close = sim->close_requested;
    *snapshot = sim->snapshot_requested;
    sim->snapshot_requested = 0;
    pthread_mutex_unlock(&sim->control);
}
```

Этот поток принимает сигналы операционной системы и записывает соответствующие запросы в структуру `Simulation`.

### Добавление `snapshot_locked`

```c
/**
 * @brief Prints the live status box as one block (world mutex locked)
 */
static void snapshot_locked(Simulation *sim)
{
    output_begin(sim);
    show_snapshot(sim);
    output_end(sim);
}
```

Эта функция защищает данные `snapshot`.

### Добавление `spawn_random_call()`

```c
/**
 * @brief Creates a random call and starts its thread (world mutex locked)
 * @return Id of the new call, or -1 if it could not be created
 */
static int spawn_random_call(Simulation *sim)
{
    int idx = create_random_call(sim);

    schedule_next_random_call(sim);

    if (idx < 0)
        return -1;

    if (!start_call_thread(sim, idx))
    {
        sim->calls[idx].state = CALL_UNSERVED;
        sim->received++;
        sim->unserved++;
        log_event(sim, "CALL #%03d REJECTED: no thread could be created, recorded as unserved", sim->calls[idx].id);
        return -1;
    }

    return sim->calls[idx].id;
```

Теперь каждому вызову необходим поток.

Поэтому последовательность стала такой:
1. Создать случайный вызов.
2. Запланировать следующий случайный вызов.
3. Проверить, удалось ли создать вызов.
4. Создать поток для нового вызова.
5. Если поток не создался, корректно зарегистрировать вызов как необслуженный.

**Добавление `deliver()`**

```c
/**
 * @brief Delivers one event to the thread that owns it (no mutex held)
 * @return 1 if a message was sent
 */
static int deliver(Simulation *sim, const Event *event)
{
    if (event->type == EV_TEAM_ARRIVAL || event->type == EV_SERVICE_DONE || event->type == EV_RETURN_DONE)
    {
        pthread_mutex_lock(&sim->world);
        int idx = find_team_index_by_id(sim, event->id);
        pthread_mutex_unlock(&sim->world);

        if (idx < 0)
            return 0;

        send_message(sim, &sim->teams[idx].mailbox, MSG_WAKE, event->type, idx);
        return 1;
    }

    // events of a call; a closed call ignores it, but its thread still reads the box
    pthread_mutex_lock(&sim->world);
    int idx = find_call_index_by_id(sim, event->id);
    int alive = idx >= 0 && sim->calls[idx].started;
    pthread_mutex_unlock(&sim->world);

    if (!alive)
        return 0;

    send_message(sim, &sim->calls[idx].mailbox, MSG_WAKE, event->type, idx);
    return 1;
}
```

Эта функция отвечает за маршрутизацию событий. Она определяет владельца события и отправляет ему `MSG_WAKE`.

**Изменение `run_smulation()`**

Суть цикла теперь такая:

1. Получаем ближайшее время;
2. Извлекаем события этого времени;

3. Отправляем события владельцам;
4. Ждём завершения обработки;
5. Отправляем диспетчеру `MSG_ROUND`;
6. Ждём назначения;
7. Проверяем инварианты;
8. Переходим к следующему времени.

## Новый файл `threads.c`

### Функция `send_message()`

```c
/**
 * @brief Sends a message and registers it in the Quiet counter
 */
void send_message(Simulation *sim, Mailbox *box, MsgType type, int event, int arg)
{
    Message message = {type, event, arg};

    quiet_add(&sim->quiet);
    mailbox_put(box, message);
}
```

Это функция отправки сообщения другому потоку.

### Функция `send_shutdown()`

```c
/**
 * @brief Sends MSG_SHUTDOWN to a thread
 */
static void send_shutdown(Mailbox *box)
{
    Message message = {MSG_SHUTDOWN, 0, 0};
    mailbox_put(box, message);
}
```

Отправляет `MSG_SHUTDOWN` потоку.

**Функция `create_thread()`**

```c
/**
 * @brief Creates a thread with a small stack
 */
static int create_thread(pthread_t *thread, void *(*body)(void *), void *arg)
{
    pthread_attr_t attr;

    if (pthread_attr_init(&attr) != 0)
        return 0;

    pthread_attr_setstacksize(&attr, THREAD_STACK_SIZE);

    int result = pthread_create(thread, &attr, body, arg);

    pthread_attr_destroy(&attr);

    if (result != 0)
    {
        fprintf(stderr, "pthread_create: %s\n", strerror(result));
        return 0;
    }

    return 1;
}
```

Создает новый поток со стаком.

**Функция `init_thread_sync()`**

```c
/**
 * @brief Initializes mutexes, the Quiet counter and the mailboxes
 * @return 1 on success, 0 on failure
 */
int init_thread_sync(Simulation *sim)
{
    SOFT_ASSERT(sim, "simulation is NULL", 0);

    SOFT_ASSERT(pthread_mutex_init(&sim->world, NULL) == 0, "cannot init world mutex", 0);
    SOFT_ASSERT(pthread_mutex_init(&sim->output_lock, NULL) == 0, "cannot init output mutex", 0);
    SOFT_ASSERT(pthread_mutex_init(&sim->control, NULL) == 0, "cannot init control mutex", 0);
    SOFT_ASSERT(quiet_init(&sim->quiet), "cannot init quiet counter", 0);
    SOFT_ASSERT(mailbox_init(&sim->dispatcher_box, DISPATCHER_BOX_SIZE), "cannot init dispatcher mailbox", 0);

    for (int idx = 0; idx < sim->team_count; idx++)
        SOFT_ASSERT(mailbox_init(&sim->teams[idx].mailbox, ENTITY_BOX_SIZE), "cannot init team mailbox", 0);

    return 1;
}
```

Инициализирует все объекты синхронизации, необходимые для работы многопоточной системы.

**Функция `destroy_thread_sync()`**

```c
/**
 * @brief Destroys all synchronization objects (threads must be joined)
 */
void destroy_thread_sync(Simulation *sim)
{
    if (sim == NULL)
        return;

    for (int idx = 0; idx < sim->call_count; idx++)
        mailbox_destroy(&sim->calls[idx].mailbox);

    for (int idx = 0; idx < sim->team_count; idx++)
        mailbox_destroy(&sim->teams[idx].mailbox);

    mailbox_destroy(&sim->dispatcher_box);
    quiet_destroy(&sim->quiet);
    pthread_mutex_destroy(&sim->control);
    pthread_mutex_destroy(&sim->output_lock);
    pthread_mutex_destroy(&sim->world);
}
```

Уничтожает созданные мьютексы, семафоры, условные переменные и почтовые ящики, освобождая связанные ресурсы.

**Функция `start_service_threads()`**

Создает и запускает основные служебные потоки симуляции.

К ним относятся:

- поток диспетчера;
- потоки бригад;
- поток обработки сигналов.

**Функция `start_call_thread()`**

```c
/**
 * @brief Creates the mailbox and the thread of a call
 * @param sim Pointer to the simulation structure
 * @param call_idx Index of the call
 * @return 1 on success, 0 on failure
 */
int start_call_thread(Simulation *sim, int call_idx)
{
    SOFT_ASSERT(sim, "simulation is NULL", 0);

    Call *call = &sim->calls[call_idx];

    SOFT_ASSERT(mailbox_init(&call->mailbox, ENTITY_BOX_SIZE), "cannot init call mailbox", 0);

    ThreadArgs *args = malloc(sizeof(*args));
    if (args == NULL)
    {
        mailbox_destroy(&call->mailbox);
        return 0;
    }

    args->sim = sim;
    args->index = call_idx;

    if (!create_thread(&call->thread, call_thread, args))
    {
        free(args);
        mailbox_destroy(&call->mailbox);
        return 0;
    }

    call->started = 1;
    sim->threads_started++;

    return 1;
}
```

Создаёт отдельный поток для обработки конкретного вызова.

**Функция `join_call_thread()`**

```c
/**
 * @brief Ends the thread of a closed call and frees its mailbox
 * @return 1 on success
 */
int join_call_thread(Simulation *sim, int call_idx)
{
    Call *call = &sim->calls[call_idx];

    if (!call->started)
        return 1;

    send_shutdown(&call->mailbox);
    pthread_join(call->thread, NULL);
    mailbox_destroy(&call->mailbox);
    call->started = 0;

    return 1;
}
```

Ожидает завершения конкретного потока вызова.

**Функция `stop_all_threads()`**

Организует завершение работы всех потоков симуляции. Функция выставляет необходимые управляющие флаги, отправляет сообщения для пробуждения потоков, которые могут находиться в ожидании, и дожидается завершения их работы.

## Файл `call_threads.c`

Файл отвечает за обработку отдельных экстренных вызовов.

Каждый вызов получает собственный поток, который ожидает сообщения и реагирует на события, связанные с его жизненным циклом.

**Функция `call_on_event()`**

Вспомогательная функция обработки события конкретного вызова.

Она вызывается из call_thread() и определяет, какое действие необходимо выполнить в зависимости от типа события.

- EV_CALL_ARRIVAL - обработка поступления вызова;
- EV_ESCALATION - повышение приоритета;
- EV_CANCEL - отмена вызова;
- EV_WAIT_TIMEOUT - обработка истечения времени ожидания.

**Функция `call_thread()`**

Функция получает аргументы потока, извлекает из них указатель на симуляцию и информацию о конкретном вызове.

Далее поток выполняет цикл обработки сообщений.

Основная последовательность:
1. Получает сообщение из почтового ящика вызова;
2. Проверяет тип полученного сообщения;
3. Обрабатывает соответствующее событие;
4. Обновляет состояние вызова;
5. Уведомляет механизм Quiet о завершении обработки сообщения;
6. Продолжает ожидание следующего сообщения либо завершает работу.

## Файл `dispatcher_thread.c`

**Функция `dispatcher_thread()`**

Главная функция потока диспетчера.

Получает аргументы потока, извлекает указатель на симуляцию и переходит в цикл ожидания сообщений.

Основная последовательность:
1. Ожидает сообщение в почтовом ящике диспетчера;
2. Получает сообщение о необходимости выполнить очередной раунд распределения;

3. Проверяет актуальное состояние симуляции;
4. Запускает алгоритм распределения `plan_dispatch()`;
5. Определяет, какие бригады можно назначить на ожидающие вызовы;
6. Отправляет выбранным бригадам сообщения `MSG_ASSIGN`;
7. Уведомляет `Quiet` о завершении обработки сообщения.

## Файл `signal_threads.c`

**Функция `fill_signall_set()`**

```c
/**
 * @brief Fills the set of signals handled by the signal thread
 */
static void fill_signal_set(sigset_t *set)
{
    sigemptyset(set);
    sigaddset(set, SIGINT);  // Ctrl+C
    sigaddset(set, SIGTERM); // kill
    sigaddset(set, SIGQUIT); // Ctrl+\
    sigaddset(set, SIGUSR1); // live status
    sigaddset(set, SIGUSR2); // internal: end of the signal thread
}
```

Формирует набор сигналов, которые должна обрабатывать программа.

В набор включаются управляющие сигналы, например:

- SIGINT;
- SIGTERM;
- SIGQUIT;
- SIGUSR1;
- SIGUSR2.

**Функция `block_signals`**

```c
/**
 * @brief Blocks the handled signals in the calling thread
 * @return 1 on success
 */
int block_signals(void)
{
    sigset_t set;
    fill_signal_set(&set);

    int result = pthread_sigmask(SIG_BLOCK, &set, NULL);
    SOFT_ASSERT(result == 0, "pthread_sigmask failed", 0);

    return 1;
}
```

Блокирует сигналы, которые должны обрабатываться специальным потоком.

**Функция `signal_thread()`**

Главная функция потока обработки сигналов.

В зависимости от полученного сигнала функция изменяет управляющее состояние симуляции:

- `SIGINT`- запрос остановки симуляции;
- `SIGTERM` - запрос остановки симуляции;
- `SIGQUIT` - прекращение приема новых вызовов;
- `SIGUSR1` - запрос создания снимка состояния;
- `SIGUSR2` - пробуждение потока для завершения.

## Файл `team_thread.c`

**Функция `team_thread()`**

Главная функция потока бригады.

Основная последовательность:

1. Ожидает сообщение в почтовом ящике бригады;
2. Получает назначение от диспетчера;
3. Обрабатывает назначенный вызов;
4. Выполняет необходимые действия, связанные с перемещением;
5. Обрабатывает прибытие на место происшествия;
6. Выполняет обслуживание;
7. Обрабатывает завершение обслуживания;
8. При необходимости возвращается на базу;
9. Уведомляет Quiet после завершения обработки сообщения.

## Файл `sync_sem.c`

### Функция `sem_wait_retry()`

```c
/**
 * @brief sem_wait() that is repeated if it was interrupted by a signal
 * @param sem Semaphore
 */
static void sem_wait_retry(sem_t *sem)
{
    while (sem_wait(sem) != 0 && errno == EINTR)
        ;
}
```

`sem_wait()` может прерваться сигналом. В таком случае мы повторяем ожидание. Эта функция для безопасного ожидания семафора.

**Функция `mailbox_init()`**

```c
int mailbox_init(Mailbox *box, int capacity)
{
    SOFT_ASSERT(box, "mailbox is NULL", 0);
    SOFT_ASSERT(capacity > 0, "mailbox capacity must be positive", 0);

    box->items = calloc((size_t)capacity, sizeof(Message));
    SOFT_ASSERT(box->items, "cannot allocate mailbox", 0);

    box->capacity = capacity;
    box->head = 0;
    box->count = 0;

    if (pthread_mutex_init(&box->lock, NULL) != 0)
    {
        free(box->items);
        return 0;
    }

    if (sem_init(&box->filled, 0, 0) != 0)
    {
        pthread_mutex_destroy(&box->lock);
        free(box->items);
        return 0;
    }

    if (sem_init(&box->free, 0, (unsigned)capacity) != 0)
    {
        sem_destroy(&box->filled);
        pthread_mutex_destroy(&box->lock);
        free(box->items);
        return 0;
    }

    return 1;
}
```

Создает и инициализирует почтовый ящик - ограниченную очередь сообщений для взаимодействия между потоками.

**Функция `mailbox_put()`**

```c
int mailbox_put(Mailbox *box, Message message)
{
    SOFT_ASSERT(box, "mailbox is NULL", 0);

    sem_wait_retry(&box->free); // wait for a free place

    pthread_mutex_lock(&box->lock);
    box->items[(box->head + box->count) % box->capacity] = message;
    box->count++;
    pthread_mutex_unlock(&box->lock);

    sem_post(&box->filled); // one more message for the consumer

    return 1;
}
```

Добавляет сообщение в почтовый ящик, ожидая появления свободного места, если очередь заполнена.

**Функция `mailbox_get()`**

```c
Message mailbox_get(Mailbox *box)
{
    Message message = {MSG_SHUTDOWN, 0, 0};

    SOFT_ASSERT(box, "mailbox is NULL", message);

    sem_wait_retry(&box->filled); // wait for a message

    pthread_mutex_lock(&box->lock);
    message = box->items[box->head];
    box->head = (box->head + 1) % box->capacity;
    box->count--;
    pthread_mutex_unlock(&box->lock);

    sem_post(&box->free);

    return message;
}
```

Извлекает сообщение из почтового ящика, ожидая появления нового сообщения, если очередь пуста.

**Функция `quiet_init()`**

```c
int quiet_init(Quiet *quiet)
{
    SOFT_ASSERT(quiet, "quiet is NULL", 0);

    quiet->pending = 0;

    if (pthread_mutex_init(&quiet->lock, NULL) != 0)
        return 0;

    if (sem_init(&quiet->idle, 0, 0) != 0)
    {
        pthread_mutex_destroy(&quiet->lock);
        return 0;
    }

    return 1;
}
```

Инициализирует счетчик незавершенных сообщений и семафор для ожидания завершения их обработки.

**Функция `quiet_destroy()`**

```c
void quiet_destroy(Quiet *quiet)
{
    if (quiet == NULL)
        return;

    sem_destroy(&quiet->idle);
    pthread_mutex_destroy(&quiet->lock);
}
```

Уничтожает семафор и мьютекс счётчика незавершенных сообщений.

**Функция `quiet_add()`**

```
void quiet_add(Quiet *quiet)
{
    pthread_mutex_lock(&quiet->lock);
    quiet->pending++;
    pthread_mutex_unlock(&quiet->lock);
}
```

Увеличивает счетчик незавершенных сообщений при отправке нового сообщения.

**Функция `quiet_done()`**

```
void quiet_done(Quiet *quiet)
{
    pthread_mutex_lock(&quiet->lock);
    quiet->pending--;

    if (quiet->pending == 0)
        sem_post(&quiet->idle);

    pthread_mutex_unlock(&quiet->lock);
}
```

Уменьшает счетчик незавершенных сообщений после обработки сообщения и сигнализирует, когда все сообщения обработаны.

**Функция `quiet_wait()`**

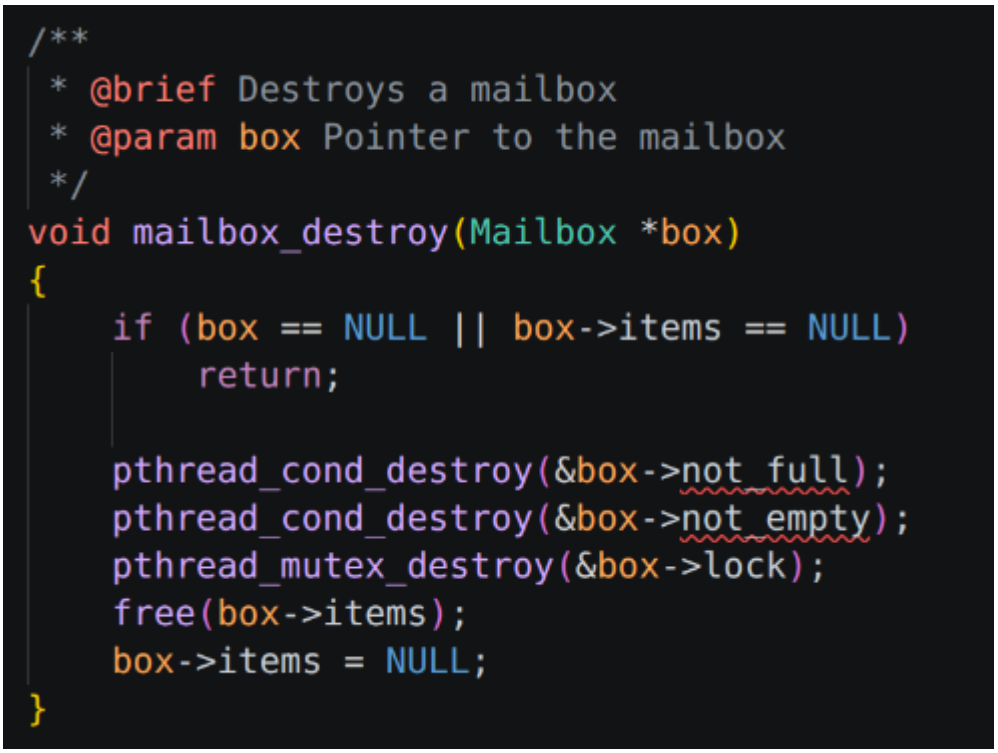
```
void quiet_wait(Quiet *quiet)
{
    sem_wait_retry(&quiet->idle);
}
```

Ожидает сигнала о завершении обработки всех сообщений.

Файл `sem_cv.c`

**Функция `mailbox_init()`**
Создает и инициализирует почтовый ящик — ограниченную очередь сообщений для взаимодействия между потоками.

**Функция `mailbox_destroy()`**



```c
/**
 * @brief Destroys a mailbox
 * @param box Pointer to the mailbox
 */
void mailbox_destroy(Mailbox *box)
{
    if (box == NULL || box->items == NULL)
        return;

    pthread_cond_destroy(&box->not_full);
    pthread_cond_destroy(&box->not_empty);
    pthread_mutex_destroy(&box->lock);
    free(box->items);
    box->items = NULL;
}
```

Освобождает память и уничтожает условные переменные и мьютекс почтового ящика.

**Функция `mailbox_put()`**

```c
/**
 * @brief Puts a message into a mailbox
 * @param box Pointer to the mailbox
 * @param message Message to put
 * @return 1 on success, 0 on failure
 */
int mailbox_put(Mailbox *box, Message message)
{
    SOFT_ASSERT(box, "mailbox is NULL", 0);

    pthread_mutex_lock(&box->lock);

    while (box->count == box->capacity) // the loop protects from spurious wake-ups
        pthread_cond_wait(&box->not_full, &box->lock);

    box->items[(box->head + box->count) % box->capacity] = message;
    box->count++;

    pthread_cond_signal(&box->not_empty);
    pthread_mutex_unlock(&box->lock);

    return 1;
}
```

Добавляет сообщение в почтовый ящик, ожидая освобождения места, если очередь заполнена.

**Функция `mailbox_get()`**

```
/**
 * @brief Gets a message from a mailbox
 * @param box Pointer to the mailbox
 * @return The message
 */
Message mailbox_get(Mailbox *box)
{
    Message message = {MSG_SHUTDOWN, 0, 0};

    SOFT_ASSERT(box, "mailbox is NULL", message);

    pthread_mutex_lock(&box->lock);

    while (box->count == 0)
        pthread_cond_wait(&box->not_empty, &box->lock);

    message = box->items[box->head];
    box->head = (box->head + 1) % box->capacity;
    box->count--;

    pthread_cond_signal(&box->not_full);
    pthread_mutex_unlock(&box->lock);

    return message;
```

Извлекает сообщение из почтового ящика, ожидая появления нового сообщения, если очередь пуста.

**Функция `quiet_init()`**

```c
/**
 * @brief  Quietly initializes the Quiet counter
 * @param quiet Pointer to the Quiet counter
 * @return 1 on success, 0 on failure
 */
int quiet_init(Quiet *quiet)
{
    SOFT_ASSERT(quiet, "quiet is NULL", 0);

    quiet->pending = 0;

    if (pthread_mutex_init(&quiet->lock, NULL) != 0)
        return 0;

    if (pthread_cond_init(&quiet->zero, NULL) != 0)
    {
        pthread_mutex_destroy(&quiet->lock);
        return 0;
    }

    return 1;
}
```

инициализирует счетчик незавершенных сообщений и условную переменную для ожидания завершения их обработки.

**Функция `quiet_destroy()`**

```c
/**
 * @brief Quietly destroys the Quiet counter
 * @param quiet Pointer to the Quiet counter
 */
void quiet_destroy(Quiet *quiet)
{
    if (quiet == NULL)
        return;

    pthread_cond_destroy(&quiet->zero);
    pthread_mutex_destroy(&quiet->lock);
}
```

Уничтожает условную переменную и мьютекс счетчика незавершенных сообщений.

**Функция `quiet_add()`**

```c
/**
 * @brief Quietly increments the pending counter
 * @param quiet Pointer to the Quiet counter
 */
void quiet_add(Quiet *quiet)
{
    pthread_mutex_lock(&quiet->lock);
    quiet->pending++;
    pthread_mutex_unlock(&quiet->lock);
}
```

Увеличивает счетчик незавершенных сообщений при отправке нового сообщения.

**Функция `quiet_done()`**

```c
/**
 * @brief Quietly decrements the pending counter and signals if it drops to zero
 * @param quiet Pointer to the Quiet counter
 */
void quiet_done(Quiet *quiet)
{
    pthread_mutex_lock(&quiet->lock);
    quiet->pending--;

    if (quiet->pending == 0)
        pthread_cond_broadcast(&quiet->zero);

    pthread_mutex_unlock(&quiet->lock);
}
```

уменьшает счетчик незавершенных сообщений и будит ожидающие потоки, когда все сообщения обработаны.

**Функция `quiet_wait()`**

```c
/**
 * @brief Waits until the pending counter drops to zero
 * @param quiet Pointer to the Quiet counter
 */
void quiet_wait(Quiet *quiet)
{
    pthread_mutex_lock(&quiet->lock);

    while (quiet->pending > 0)
        pthread_cond_wait(&quiet->zero, &quiet->lock);

    pthread_mutex_unlock(&quiet->lock);
}
```

Ожидает, пока счетчик незавершенных сообщений не станет равен нулю.

## Изменение `main.c`

```c
if (!start_service_threads(&sim))
{
    destroy_thread_sync(&sim);
    close(sim.log_fd);
    return 1;
}

// one thread for every configured call (the caller)
for (int idx = 0; idx < sim.call_count; idx++)
{
    if (!start_call_thread(&sim, idx))
    {
        fprintf(stderr, "cannot create the thread of call #%d\n", sim.calls[idx].id);
        stop_all_threads(&sim);
        destroy_thread_sync(&sim);
        close(sim.log_fd);
        return 1;
    }
}

int ok = run_simulation(&sim); // the main thread works as the clock

stop_all_threads(&sim);

// all threads are joined, the summary sees the final state
print_summary(&sim);

destroy_thread_sync(&sim);
```

Теперь сначала блокируем сигналы, а затем инициализируем средства синхронизации.

Мы должны запустить:

- диспетчера;
- бригады;
- поток сигналов.

Если создание потоков не удалось, программа завершает подготовку и освобождает ресурсы.

В конце сначала дожидаемся окончания потоков, потом выводим окончательную статистику и уничтожаем средства синхронизации.